

Konspekt prelegenta — Techniki optymalizacyjne w JS

Benchmarki: liczby orientacyjne (Node 22 / Chrome, mogą się różnić między maszynami).

Zasada na każdy slajd: powiedz CLUE własnymi słowami → 2–3 punkty → benchmark/przykład → zdanie przejścia.

02 · Wstęp

CLUE: Najpierw zmierz, potem optymalizuj. Największe zyski daje algorytm i struktura danych — mikro-optymalizacje pod JIT tylko w zmierzonych hot pathach.

- Lewa kolumna: co daje najwięcej (złożoność, DOM/reflow, `Promise.all`, stałe kształty obiektów).
- Prawa: gdzie ucieka pamięć (listeners/timers, cache bez limitu, detached DOM).
- Knuth: 97% kodu nie optymalizuj — **ale ta prezentacja jest o krytycznych 3%**.

Przejście: „Żeby rozumieć te 3%, trzeba wiedzieć, jak V8 zarządza pamięcią...”

03 · Pamięć i GC

CLUE: GC nie liczy referencji — pyta „czy da się do mnie dojść od rootów?” (globalThis, call stack, timers, listeners). Wyciek = pamięć osiągalna przez przypadek.

- Stos: ramki + małe wartości; mit „prymitywy na stosie” — string/HeapNumber żyją na stercie. Wycieki zawsze dotyczą sterty.
- Diagram: **hipoteza generacyjna** — większość obiektów umiera młodo. YOUNG = Scavenge (kopiuje tylko żywe, pauza 1–5 ms), przeżył 2 cykle → promocja do OLD (Mark-Sweep-Compact, rzadszy, inkrementalny).
- **Pułapka:** obiekt przytrzymany przez listener/cache awansuje do OLD i obciąża drogi kolektor do końca życia aplikacji.
- Tabelka top-5 wycieków: timer → `clearInterval`, listener → `AbortController`, detached node → nulluj, cache → LRU/WeakMap, globalne → moduły ES.

Zamknięcie: „GC zwalnia was z `free()`, nie z myślenia — co trzymacie osiągalne, to wasze.”

04 · Kompilacja V8

CLUE: V8 kompiluje tyle, ile trzeba — funkcja wywołana raz zostaje w interpreterze, gorący kod wspina się po tierach. Im wyższy tier, tym więcej spekulacji o typach — i tym więcej do stracenia.

- Pipeline: Ignition (bytecode + **feedback o typach**) → Sparkplug → Maglev → TurboFan (pełna optymalizacja spekulatywna).
- Założenie pada (inny typ) → **deopt**: kod wyrzucony, powrót do bytecode. Diagnostyka: `node --trace-deopt`.
- Dobre wieści: kompilacja w tle, OSR podmienia gorącą pętlę w locie.

Wniosek: stabilne typy + stałe kształty = kod zostaje na najwyższym tierze. Resztą zajmuje się V8.

05 · Liczby

CLUE: Dla programisty jeden typ `number` (64-bit double IEEE 754). Dla V8 dwa światy: **Smi** (mały int w słowie wskaźnika, zero alokacji) i **HeapNumber** (pudełko na stercie dla reszty).

- IEEE 754: bezpieczne inty do $2^{53}-1$ (`253 === 253+1` → true!), `0.1+0.2 !== 0.3`, `NaN !== NaN`.
- Pętle po małych intach = wszystko w Smi = najszybciej.
- Pomost do tablic: jeden `push(0.5)` przepisuje tablicę intów na double — nieodwracalnie.

06 · Stringi [bench]

CLUE: String to nie zawsze płaski bufor: `a + b` = rope (para wskaźników, O(1)), `substring` = widok na rodzica.

- **Mit „join zamiast +=” jest martwy** — bench: `+=` ~2× szybsze (rope; flatten odroczone do pierwszego odczytu — dlatego mierzymy z odczytem znaku!).
- **Pułapka substring:** 64-znakowy nagłówek z 10 MB pliku w cache’u = 10 MB w pamięci. Wycinek ma żyć długo → wymuś kopię.
- Bonus: literały/klucze internalizowane (porównanie wskaźnikiem O(1)); jeden „ż” przetacza cały string na 2 B/znak.

Bench: `+=` wygrywa ~×2.

07 · Tablice — elements kinds

CLUE: V8 śledzi, co tablica trzyma w środku, i dobiera reprezentację. Przejścia **jednokierunkowe** — jeden zły push degraduje na zawsze.

- Krata: PACKED_SMI → PACKED_DOUBLE → PACKED_ELEMENTS; dziura spycha w dół do HOLEY_*. Nigdy w lewo, nigdy w górę.
- Holey: każdy odczyt sprawdza „czy dziura?” — a dziura może mieć wartość z `Array.prototype`, stąd chodzenie po prototypach.
- Reguły: nie `new Array(N)` (→ `Array.from`), nie `delete arr[i]` (→ `splice`), jednorodny typy.
- Demo w kodzie: `%DebugPrint` w `node --allow-natives-syntax`.

Przejście: „Za chwilę to zmierzymy — najpierw dwa klasyki struktur danych.”

08 · Set vs Array [bench x2]

CLUE: `includes` skanuje liniowo $O(n)$, `has` liczy hash $O(1)$. Częste sprawdzanie przynależności / deduplikacja → Set wygrywa o rzędy wielkości.

- Array kontruje: ciągła pamięć, świetna iteracja, dla $< \sim 100$ elementów bywa szybszy.
- **Bench 1** (1000 zapytań $\times$ 100k elementów): Set szybszy $\sim \times 100$ –1000.
- **Bench 2** (deduplikacja 50k): `[...new Set(arr)]` $O(n)$ vs `filter+indexOf` $O(n^2)$ — przepaść.

09 · Map vs Object [bench x2]

CLUE: Object jest zoptymalizowany pod stałą strukturę (hidden class + IC = dostęp jak struct w C). `delete` i dynamiczne klucze wybijają go do dictionary mode. Map jest słownikiem z założenia — nie ma skąd spaść.

- **Bench 1** (rotacja kluczy z delete): Map szybszy kilka $\times$; podkreśl — to `delete` zabija hidden class (w kodzie zwykle `{}`, nie `Object.create(null)` — ten byłby słownikiem od startu).
- **Bench 2** (iteracja 50k kluczy): `for...of` po Mapie vs `Object.keys` (alokacja tablicy!) — Map ~ 2 – $3 \times$.
- Kontra: stały zestaw kluczy + częsty odczyt → Object szybszy i lżejszy.

10 · Elements kinds w praktyce [bench – tabela 7 wariantów]

CLUE: Ta sama pętla odczytu, 7 wariantów tej samej tablicy — różnica wyłącznie z wewnętrznej reprezentacji.

- Oczekiwane: PACKED_SMI $1 \times$ → HOLEY_SMI $\sim 2 \times$ → PACKED_ELEMENTS $\sim 4 \times$ → DICTIONARY $\sim \text{setki}$ – $1000 \times$ (uczciwie: ma $20 \times$ dłuższą pętlę po dziurach — notka pod tabelą).
- Smaczek metodologiczny (jak ktoś spyta): każdy wariant ma **osobną kopię funkcji odczytu** — wspólna miałaby generyczny IC i zatarała różnice. Sekwencyjne `a[i*10]` NIE robi dictionary — trzeba skoku > 1024 ponad length.

11 · Hidden classes [bench – tabela]

CLUE: V8 opisuje kształt obiektu ukrytą klasą; obiekty o tym samym kształcie ją współdzielą, a inline cache zamienia `obj.x` w odczyt spod stałego offsetu. Im więcej kształtów widzi jedno miejsce w kodzie, tym wolniej.

- Drabinka: monomorphic (1 kształt, cel) → polymorphic (2–4, stub z listą) → megamorphic (> 4 , generyczny lookup, koniec inliningu — **ale to nie deopt**).
- Co niszczy kształt: inna kolejność pól (`{x,y}` $\neq$ `{y,x}`), pola po konstrukcji, `delete` (→ `null`).
- **Bench:** mono vs poly vs mega (1/4/32 HC) — mega ~ 2 – $3 \times$ wolniejszy na tych samych polach `.x .y`.

12 · Deoptymalizacja [bench]

CLUE: TurboFan kompiluje pod kształty, które widział. Nowy kształt = złamane założenie = deopt (kod wyrzucony, powrót do interpretera). Megamorfizm $\neq$ deopt — to utrata specjalizacji, kod dalej działa.

- **Bench:** ta sama funkcja `distance(p)`, argumenty o 1 / 4 / 6 kształtach — mega vs mono $\sim \times 2$ – 5 .
- Reguły: jedna funkcja = jeden kształt argumentu; nie mieszaj typów (rozdziel funkcje); unikaj `delete`.

13 · WeakMap / WeakRef [demo]

CLUE: Klucz w Map trzyma obiekt przy życiu — cache na Mapie to gotowy wyciek. WeakMap trzyma klucz słabo: kod zapomni o obiekcie → GC zbiera go razem z wpisem.

- Brak iteracji i `.size` — to celowe (rozmiar z definicji nieznan).
- **Demo:** FinalizationRegistry liczy zebrane obiekty. UWAGA: niedeterministyczne — jak pokaże 0, powiedz to głośno (to feature, nie bug) i odpał jeszcze raz / wymuś GC w DevTools.
- Kiedy nie: potrzebna iteracja/rozmiar → zwykły Map + jawne sprzątanie.

14 · Klonowanie [bench x2]

CLUE: Wybór metody = kompromis szybkość vs poprawność. Płytko → spread, głęboko → structuredClone, JSON tylko dla danych czysto JSON-owych.

- Tabela: spread (najszybszy, płytki, gubi prototyp/gettery) · assign (j.w. + scalanie) · JSON (gubi typy, **cykl = wyjątek**) · structuredClone (Date/Map/Set/cykle ok; funkcje = wyjątek).
- **Bench 1** płytko: spread $\geq$ assign $\gg$ JSON ($\sim \times 10$).
- **Bench 2** głęboko: JSON często SZYBSZY od structuredClone — puenta: szybszy $\neq$ lepszy, wygrywa gubiąc typy.
- Najczęstszy błąd w praktyce: płytka kopia tam, gdzie ktoś zakładał głęboką (`clone.nested.x` mutuje oryginał).

15 · TypedArray

CLUE: ArrayBuffer = surowy blok bajtów: zero tagowania, zero nagłówek per element, jeden węzeł dla GC. Dla dużych jednorodnych danych liczbowych 4 – $32 \times$ mniej pamięci.

- Tabela: Array OBJECT_ELEMENTS ~ 32 B/element → Float64Array 8 B → Uint8Array 1 B.
- `subarray()` = widok bez kopii $O(1)$ (vs `slice` Array = zawsze kopia $O(n)$).
- Pułapka: jedno `push(null)` na tablicy liczb → OBJECT_ELEMENTS, pamięć $\times 4$.

16 · Object pooling [bench]

CLUE: Zamiast tworzyć i porzucać tysiące obiektów na klatkę — alokuj raz, reużywaj. Zero alokacji w hot path = GC nie ma czego zbierać (60 fps: jedna pauza GC = zgubiona klatka).

- Bonus: wszystkie obiekty z puli mają ten sam kształt → mono IC.
- **Bench** (10k cząsteczek × 500 klatek): pool ~×2–5.
- **WAŻNE zastrzeżenie (przeczytaj tip!):** pooling bywa szkodliwy — pula trzyma obiekty przy życiu → awansują do old generation. Tylko zmierzone, wrażliwe na pauzy hot pathy (gry, symulacje, audio).

17 · Web Workers

CLUE: Worker ma własną stertę i własny GC — ciężka robota nie blokuje UI. Haczyk: `postMessage` domyślnie KOPIUJE (structured clone). Duże bufory transferuj — $O(n)$ → $O(1)$.

- Tabela transferables: ArrayBuffer (transfer — źródło odłączone!), SharedArrayBuffer (współdzielony, wymaga Atomics + COOP/COEP), zwykły obiekt (kopia).
- Kopia 40 MB ≈ 10–30 ms + 2× pamięć; transfer = przeniesienie wskaźnika.
- Worker pool: start workera to kilka–kilkanaście ms — twórz raz, kolejkuje zadania.

18 · Profiling

CLUE: Workflow: nagraj → znajdź long task (>50 ms) → rozwiń flame chart → popraw → **zmiar ponownie**. Szukaj szerokich bloków z dużym self time — to one są wąskim gardłem, nie ich rodzice.

- Nie profiluj zimnego startu (JIT się rozgrzewa); incognito bez rozszerzeń; CPU throttling 4–6× = mobile.
- `performance.mark/measure` (widoczne w Timings), `PerformanceObserver` na long taski w produkcji.
- Pamięć: 2 heap snapshoty + Comparison, sortuj po Retained Size, filtr „Detached”. `console.log(obj)` trzyma referencję — fałszuje pomiar wycieków!

19 · Podsumowanie

CLUE (złota zasada): zmierz, zanim zoptymalizujesz — V8 zmienia się co kilka tygodni, profiler na Twojej wersji silnika nie.

- Cheat sheet: 4 panele (kształty/JIT, tablice, struktury danych, pamięć) — przeleć po boldach.
- Mity nieaktualne: try/catch, join vs +=, cache'owanie length, „małe funkcje drogie”.
- Nadal kosztuje: `delete`, zmiana kształtu w hot path, megamorfizm, **shift() jako kolejka — zmierzone: 200k shiftów = 19 s**.

Trudne pytania z sali — szybkie odpowiedzi

- „**Megamorphic to deopt?**” Nie — megamorfizm to utrata specjalizacji IC (generyczny lookup); deopt to wyrzucenie skompilowanego kodu po złamaniu konkretnego założenia o typie.
- „**Skąd ~1–16 MB young gen?**” Zależy od wersji/konfiguracji V8 (semi-spaces); rząd wielkości, nie stała.
- „**Czemu w benchmarku elements kinds DICTIONARY jest aż tak wolny?**” Bo ma też ~20× dłuższą pętlę (length 2M) — notka pod tabelą; sam tryb słownikowy to mniejszy mnożnik.
- „**A double field unboxing w obiektach?**” Usunięty z V8 w 2020 (pointer compression) — pola double to dziś wskaźnik na mutable HeapNumber; unboxed double żyją tylko w tablicach.
- „**JSON szybszy od structuredClone, to czemu go nie używać?**” Bo gubi typy i wybucha na cyklach — szybkość kosztem poprawności; ok tylko dla danych czysto JSON-owych.
- „**Czy V8 nie optymalizuje shift()?**” Mierzyłem: 200k `shift()` = 19 s — $O(n^2)$. Kolejka → wskaźnik głowy albo deque.